SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 1 – Case Study Analysis

Course Code:	CSA1017	**Course Title:**	Software Engineering
CO Assessed:	CO1	**Assessment:**	Assessment Tool 1 – Case Study Analysis
Weightage:	20%	**Total Marks:**	25
CO1 Statement:	*Apply advanced methodologies and agile project management techniques to manage software projects effectively*		
Student Name:	Guru charan M	**Reg. No.:**	192524434
Date:	20/10/2026	**Duration:**	60 minutes

Code of Conduct

I Guru charan M/192524434 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Guru charan M

Contents

Section	Title	Focus
1	Understanding the Problem Statement	Problem, objectives and scope
2	Key Stakeholders and Challenges	Roles, responsibilities and issues
3	Analysis of the Current Approach	Existing process, strengths, limitations, gaps
4	Proposed Software Solution	Modules, requirements, engineering principles
5	Methodology and Framework Justification	Agile Scrum with DevOps
6	Expected Outcomes and Limitations	Benefits, risks, future enhancements

1. Understanding the Problem Statement

1.1 The Problem in Our Own Words

Factories run on machines such as CNC machines, conveyors, motors, pumps, compressors and robotic arms. When one of these machines stops without warning, the whole production line can stop with it. Orders are delayed, workers sit idle, emergency repairs cost far more than planned ones, and customers lose trust.

Most plants today handle this in one of two ways. Either they **repair a machine after it breaks** (reactive maintenance), or they **service machines on a fixed calendar** whether or not the machine needs it (time-based preventive maintenance). The first approach causes expensive surprises. The second wastes money and labour on healthy machines and can still miss a machine that is deteriorating faster than the calendar assumes.

The task is to design a **Predictive Maintenance Platform**: a central software system that continuously watches the condition of equipment across many plants, learns what a healthy machine looks like, detects early signs of failure, and then recommends and schedules maintenance at the right time. In short, the platform shifts maintenance from "fix it when it breaks" to "fix it just before it is about to break".

1.2 Objectives of the Proposed System

Table 1: Objectives and how the platform meets them

Objective	How the platform achieves it
Monitor equipment performance	Collects sensor readings (temperature, vibration, pressure, current, speed) in near real time and computes health indicators.
Track maintenance activities	Records every work order, inspection, repair and spare part used, with a full history per asset.
Predict equipment failures	Uses trend analysis and machine-learning models to estimate failure risk and remaining useful life.
Schedule preventive maintenance	Recommends maintenance windows based on risk, production plan and technician availability.
Notify maintenance teams	Sends alerts by dashboard, email, SMS or mobile push, with escalation if no one responds.
Generate maintenance analytics	Produces reports on downtime, MTBF, MTTR, maintenance cost and compliance.
Improve equipment utilization	Tracks availability, performance and quality (OEE) to find under-used or problem assets.
Reduce production downtime	Converts unplanned stoppages into short, planned ones.
Improve manufacturing efficiency	Gives managers one view of all plants so resources go where they are needed most.
Support asset lifecycle management	Tracks each asset from installation to retirement, supporting repair-or-replace decisions.

1.3 Scope of the Problem

The scope defines what the platform will and will not cover, so that the project stays achievable.

Table 2: Project scope

In Scope	Out of Scope
Multi-plant, multi-line, multi-team asset monitoring	Replacing or controlling the machines themselves (PLC control logic)
Sensor data collection through gateways and industrial protocols	Manufacturing the sensors or edge hardware
Equipment health records and asset registry	Full ERP functions such as payroll, sales and finance
Configurable maintenance policies and KPIs	Supply-chain planning beyond spare-part availability
Failure prediction, alerts and maintenance scheduling	Fully autonomous repair without human approval
Work-order and maintenance-history tracking	Operator training programmes (only documentation is provided)
Dashboards, analytics and reports	
Role-based security, high availability and scalability	

2. Key Stakeholders and Challenges

2.1 Stakeholders, Roles and Responsibilities

A predictive maintenance platform affects people from the shop floor to the executive office. Identifying all of them early prevents missed requirements later.

Table 3: Stakeholders and their responsibilities

Stakeholder	Role and responsibilities	Main challenges faced
Plant Manager	Oversees production targets, plant performance and budget; approves maintenance windows.	Unexpected stoppages, missed deliveries, poor visibility of equipment condition.
Maintenance Manager	Plans maintenance strategy, assigns work, manages spare parts and compliance.	Unplanned workload, fixed schedules that ignore real condition, weak history records.
Maintenance Technicians	Inspect, repair and service machines; update work orders on site.	Late or unclear alerts, paper-based records, searching for machine history.
Machine Operators	Run machines daily and report abnormal sounds, heat or vibration.	No easy way to log early symptoms; often blamed for failures.
Reliability / Data Engineers	Define health indicators, thresholds and prediction models; tune accuracy.	Poor-quality or missing sensor data, false alarms, model drift.
Production Planners	Schedule production orders and line capacity.	Maintenance clashing with production; sudden capacity loss.
IT / System Administrators	Deploy, secure, back up and support the platform and integrations.	Legacy machine integration, cyber-security, uptime and data growth.
Executive Management	Set strategy; review cost, efficiency and return on investment.	High maintenance cost, no enterprise-wide analytics, unclear ROI.
Quality & Safety Officers	Ensure safe operation and regulatory compliance.	Failures that create safety incidents or defective products.
Equipment Vendors / Suppliers	Provide machines, spare parts and warranty or service support.	Limited failure data sharing; delays in parts and service.
Customers	Receive finished goods on time and at agreed quality.	Delayed deliveries and inconsistent quality caused by breakdowns.

2.2 Major Challenges and Issues

Operational challenges

1. **Unplanned downtime:** a single critical machine failure can halt an entire production line and cost thousands per hour.
2. **Over- and under-maintenance:** fixed schedules service healthy machines unnecessarily and still miss fast-developing faults.
3. **Fragmented information:** maintenance logs sit in spreadsheets, paper files and individual

memories, so history is hard to use.

Technical challenges

4. **Data quality:** sensors drift, fail or send noisy data, and different machines use different formats and protocols.
5. **Legacy equipment:** older machines have no digital interface and need retrofitted sensors and gateways.
6. **Prediction accuracy:** too many false alarms make technicians ignore alerts; too few lets failures through.
7. **Scale:** many plants and thousands of assets generate very large streams of time-series data.

Organizational and security challenges

8. **Change resistance:** staff used to run-to-failure habits may distrust automated recommendations.
9. **Cyber-security:** connecting factory machines to a network increases the attack surface.
10. **Coordination:** maintenance windows must be negotiated with production planning across plants.

3. Analysis of the Current Approach

3.1 Description of the Existing System

In most manufacturing organisations the existing process combines three practices:

11. **Reactive (breakdown) maintenance:** a machine runs until it fails; operators raise a call, and technicians repair it under time pressure.
12. **Time-based preventive maintenance:** machines are inspected or serviced at fixed intervals (for example every 500 operating hours or every quarter) taken from the manufacturer's manual.
13. **Manual record keeping:** work orders on paper, spreadsheets or a basic maintenance tool; readings are taken by walk-round inspections and entered by hand.

Information flows slowly in this model: an operator notices a problem, informs a supervisor, who informs maintenance, who may then look up an old record. Each plant tends to work independently, so management has no single, current picture across facilities.

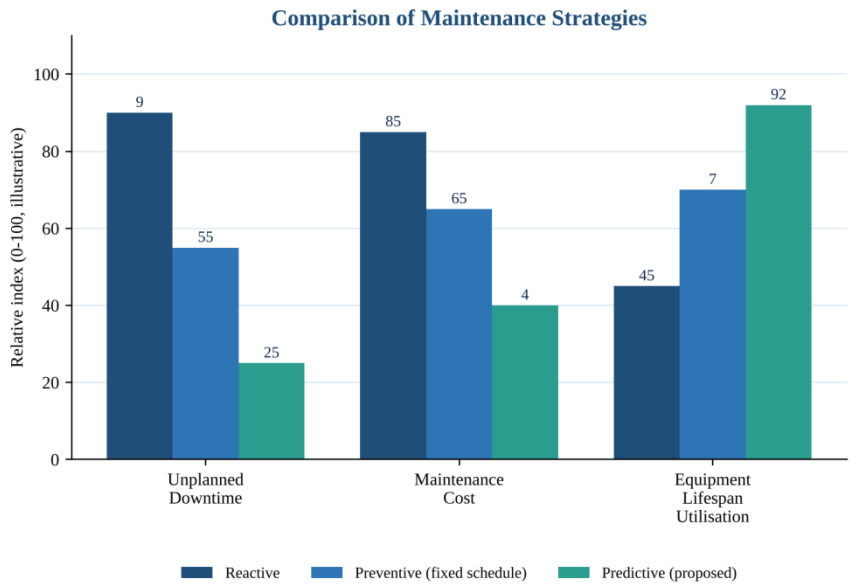


Figure 1: Indicative comparison of reactive, preventive and predictive maintenance (illustrative values)

3.2 Strengths of the Current Approach

14. **Simple and familiar:** staff already understand the routine and need little training.
15. **Low upfront cost:** no investment in sensors, software or analytics skills.
16. **Predictable calendar:** fixed schedules are easy to plan and audit for basic compliance.
17. **Manufacturer-backed:** intervals follow vendor recommendations, which helps protect warranties.

3.3 Limitations of the Current Approach

Table 4: Limitations and their business impact

Limitation	Impact
Failures are discovered only after they happen	Unplanned downtime, emergency repair cost, missed deliveries.
Schedules ignore actual machine condition	Healthy machines are serviced too often; deteriorating ones are missed.
Manual and paper-based records	Errors, lost history, slow audits and poor decision support.
No continuous monitoring	Early warning signs such as rising vibration or temperature go unnoticed.
Plants work in isolation	No enterprise-level visibility, inconsistent practices, duplicated spare stock.
Weak alerting and escalation	Slow response; critical issues may wait for the next shift.
Limited analytics	Managers cannot measure MTBF, MTTR, OEE or maintenance cost trends reliably.

3.4 Gaps That Need to Be Addressed

Table 5: Gap analysis

Gap	Current state	Required state
Condition awareness	Periodic manual checks	Continuous automated monitoring
Failure handling	React after failure	Predict and prevent before failure
Scheduling	Fixed calendar	Dynamic, risk- and production-aware
Data and records	Paper, spreadsheets, silos	Centralised, searchable asset history
Notifications	Phone calls and word of mouth	Automatic, prioritised, escalated alerts
Visibility	Per plant, delayed reports	Real-time multi-plant dashboards
Decision support	Experience and guesswork	Data-driven analytics and recommendations
Security and availability	Ad-hoc, local files	Role-based access, backups, high availability

4. Proposed Software Solution

4.1 Solution Overview

The proposed solution is a **cloud-based, modular Predictive Maintenance Platform** that connects to machines through edge gateways, streams sensor data into a central processing layer, analyses it using configurable policies and machine-learning models, and presents results through dashboards, alerts and reports. It is designed as a set of loosely coupled services so that each capability can be developed, scaled and updated independently.

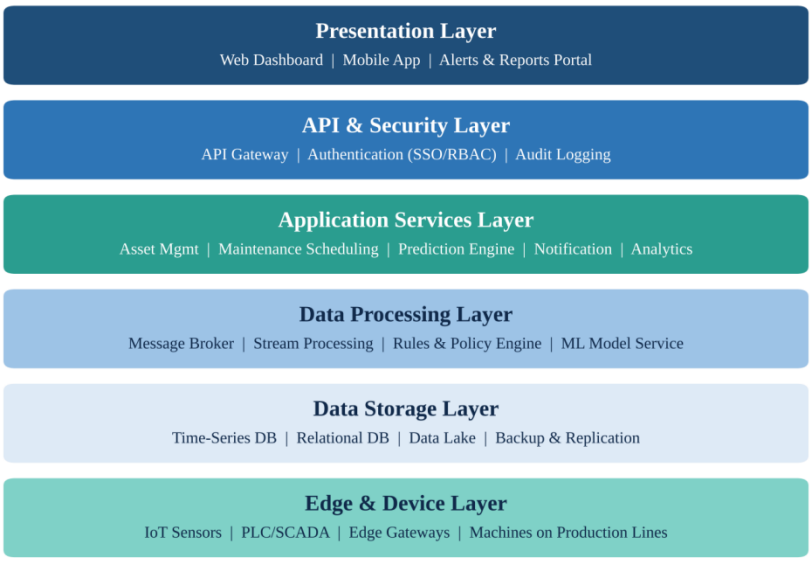


Figure 2: Layered architecture of the Predictive Maintenance Platform

The prediction workflow below shows how a raw sensor reading becomes a maintenance action.



Figure 3: End-to-end predictive maintenance workflow

4.2 Major Functional Modules

Table 6: Functional modules

Module	Description
Asset Management	Registry of plants, lines, machines and components with specifications, location, warranty, installation date and lifecycle status.
Data Acquisition & Ingestion	Connects to sensors, PLC/SCADA and gateways through industrial protocols (for example OPC UA, MQTT, Modbus); validates and buffers incoming data.
Equipment Health Monitoring	Calculates health indicators and health scores, tracks trends, and displays live machine condition and utilisation (availability, performance, OEE).
Policy & Rules Engine	Lets administrators configure thresholds, KPIs and maintenance policies per asset type without code changes.
Failure Prediction Engine	Applies anomaly detection and predictive models to estimate failure probability and remaining useful life.
Maintenance Scheduling	Recommends and books preventive maintenance windows, considering risk, production plans, technicians and spare parts.
Work-Order Management	Creates, assigns, tracks and closes work orders; records actions, time, cost and spare parts; keeps full maintenance history.
Notification & Escalation	Sends alerts through email, SMS, push and dashboard; escalates unacknowledged critical alerts.
Dashboard & Analytics	Role-based dashboards and reports: downtime, MTBF, MTTR, cost, schedule compliance and plant comparisons.
User & Security Management	Authentication, role-based access control, audit trails and encryption.
Integration Services	APIs to connect with ERP, inventory and production-planning systems.

4.3 Functional Requirements

Table 7: Functional requirements

ID	Requirement
FR1	The system shall register and manage assets across multiple plants and production lines.
FR2	The system shall ingest sensor data continuously and store it as time-series records.
FR3	The system shall calculate and display an equipment health score for every monitored asset.
FR4	The system shall allow authorised users to configure maintenance policies, thresholds and KPIs.

ID	Requirement
FR5	The system shall predict potential failures and estimate remaining useful life.
FR6	The system shall recommend preventive maintenance schedules and allow planners to approve or change them.
FR7	The system shall create, assign, track and close maintenance work orders.
FR8	The system shall maintain a complete, searchable maintenance history for each asset.
FR9	The system shall notify maintenance teams of alerts and escalate unacknowledged critical alerts.
FR10	The system shall provide role-based dashboards for operators, technicians, managers and executives.
FR11	The system shall generate and export analytics and reports (PDF, Excel).
FR12	The system shall track asset lifecycle events from installation to retirement.
FR13	The system shall integrate with ERP and inventory systems through APIs.
FR14	The system shall support multiple plants, teams and user roles with separate access.

4.4 Non-Functional Requirements

Table 8: Non-functional requirements

Category	Requirement
Performance	Dashboards shall refresh within 5 seconds; alerts shall be raised within 30 seconds of a threshold breach.
Scalability	The system shall scale horizontally to support additional plants, assets and sensors without redesign.
Availability	Target availability of at least 99.9%, with redundancy and automatic failover.
Reliability	No loss of sensor data during short network failures (edge buffering); automated backups and recovery.
Security	Encrypted data in transit and at rest, role-based access, multi-factor authentication and audit logs.
Usability	Simple, consistent interface; mobile-friendly for technicians; minimal training needed.
Maintainability	Modular code, documented APIs, automated tests and configuration over hard-coding.
Interoperability	Standard industrial protocols and open APIs for integration.
Compliance	Retain maintenance and audit records according to industry and safety regulations.

4.5 Incorporation of Software Engineering Principles

Table 9: How software engineering principles are built into the design

Principle	How it is applied in the solution
Modularity	The system is divided into independent services (asset, ingestion, prediction, scheduling, notification) with clear interfaces, so each can be built, tested and replaced separately.
Scalability	Stateless services run in containers and scale horizontally; a message broker and time-series database handle growing data volumes; new plants are added through configuration.
Maintainability	Clean layered architecture, coding standards, automated unit and integration tests, version-controlled code, and policies stored as configuration rather than code.
Reliability	Redundant servers, database replication, edge buffering, health checks, automatic restart and regular backup and recovery drills.
Usability	Role-based dashboards, colour-coded health status, mobile work-order screens and a short learning curve validated through user testing.
Separation of concerns	Presentation, business logic, data processing and storage are kept in separate layers.
Reusability & extensibility	Common services (notifications, authentication) are reused; new sensors or models plug in through defined interfaces.

4.6 Example of a Configurable Maintenance Policy

The policy engine lets the maintenance manager define rules without programming. A typical rule set for a motor might be:

Table 10: Sample policy for an industrial motor

Indicator	Warning level	Critical level	Action
Vibration (mm/s)	Above 4.5	Above 7.1	Warning: schedule inspection within 7 days. Critical: immediate work order.
Bearing temperature (°C)	Above 75	Above 90	Warning: check lubrication. Critical: stop and inspect.
Motor current (% of rated)	Above 110%	Above 125%	Warning: check load. Critical: alert on-call technician.
Predicted failure risk (30 days)	Above 40%	Above 70%	Warning: plan maintenance window. Critical: schedule at next stop.

Threshold values are illustrative; real values are set per asset type from manufacturer data and historical records.

5. Methodology and Framework Justification

5.1 Selected Methodology: Agile (Scrum) with DevOps Practices

For this platform we select **Agile using the Scrum framework, supported by DevOps practices** (continuous integration, continuous delivery, automated testing and monitoring). Work is delivered in short sprints of two to three weeks, each producing a working, tested increment that users can review.

5.2 Comparison of Candidate Models

Table 11: Evaluation of SDLC models for this project

Model	Strengths	Weaknesses for this project	Fit
Waterfall	Clear phases, strong documentation.	Requirements and prediction accuracy cannot be fixed upfront; late feedback; costly changes.	Low
Spiral	Strong risk analysis, iterative.	Heavy process and cost; slower delivery of usable features.	Medium
Agile (Scrum)	Iterative delivery, frequent feedback, adaptable to change.	Needs disciplined planning and an engaged product owner.	High
DevOps	Automation, fast reliable releases, strong operations focus.	Is a practice set rather than a full planning model.	High
Agile + DevOps (selected)	Combines adaptable development with automated, dependable deployment and monitoring.	Requires team skills in automation and tooling.	Very high

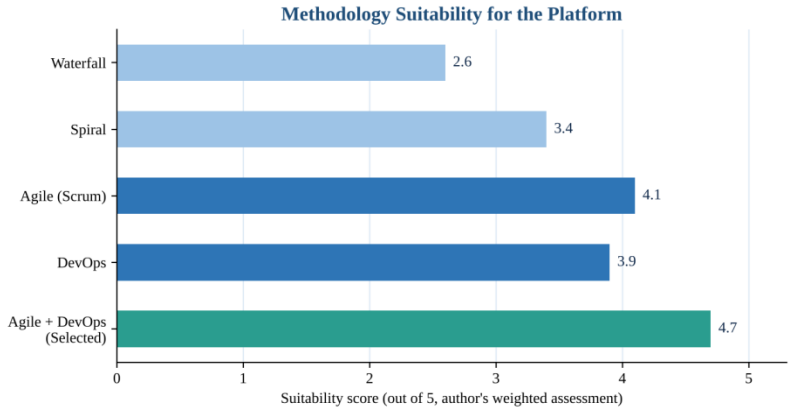


Figure 4: Suitability of methodologies (author's weighted assessment against the criteria in 5.3)

5.3 Why This Methodology Suits the Platform

18. **Evolving requirements:** failure patterns, thresholds and user needs become clear only after real data is seen. Agile accepts and plans for change.
19. **Early value:** monitoring and dashboards can go live for one pilot plant in the first releases, then prediction, scheduling and analytics follow sprint by sprint.

20. **Model improvement:** prediction models must be trained, tested and retrained as data grows; short feedback loops fit this experimental work.
21. **Stakeholder involvement:** technicians, planners and managers review each increment, which reduces change resistance and builds trust.
22. **Reliability and uptime:** the platform itself must be highly available; DevOps automation (CI/CD, automated tests, monitoring, rollback) allows frequent releases with low risk.
23. **Risk control:** integration with legacy machines and security concerns are tackled early through prototypes and spikes.

5.4 How the Methodology Supports Development and Maintenance

Table 12: Proposed release plan

Phase / Sprints	Deliverables
Sprints 1-2: Foundation	Requirements backlog, architecture, asset registry, user and role management, CI/CD pipeline.
Sprints 3-5: Data and monitoring	Sensor ingestion, time-series storage, health scores, first dashboards (pilot plant).
Sprints 6-8: Work and alerts	Work-order management, maintenance history, notifications and escalation, policy engine.
Sprints 9-11: Prediction	Anomaly detection, failure prediction, maintenance recommendation and scheduling.
Sprints 12-13: Analytics and scale	Analytics, reports, multi-plant rollout, performance and security testing.
Ongoing: Operations	Monitoring, patching, model retraining, user feedback, incremental enhancements.

During development, Scrum events (sprint planning, daily stand-up, review and retrospective) keep the team aligned and visible. **During maintenance**, the same backlog-driven cycle handles bug fixes and new features; automated pipelines, test suites and monitoring make updates fast and safe, and modular services mean a change in one area does not disturb the rest.

6. Expected Outcomes and Limitations

6.1 Expected Benefits

24. **Fewer unexpected failures** through early detection of abnormal behaviour.
25. **Better production continuity** because repairs are planned around production schedules.
26. **Lower maintenance cost** by avoiding emergency repairs and unnecessary routine servicing.
27. **Longer equipment life** because faults are corrected before they cause secondary damage.
28. **Improved efficiency and utilisation** with real-time OEE and availability insight.
29. **Better planning and visibility** through a single multi-plant view of equipment condition.
30. **Informed decisions** based on trusted history and analytics for repair-or-replace choices.
31. **Scalable operations** that extend to new plants and assets through configuration.

6.2 How the Solution Addresses the Identified Challenges

Table 13: Challenges mapped to solution features

Challenge (Section 2 and 3)	How the solution addresses it
Unplanned downtime	Continuous monitoring and failure prediction trigger action before breakdown.
Over- and under-maintenance	Condition-based, risk-driven scheduling replaces fixed calendars.
Fragmented records	Centralised asset registry, work orders and searchable history.
Slow response	Automatic prioritised alerts with escalation.
Plants working in isolation	Multi-plant dashboards and consistent policies.
Poor data quality	Validation at ingestion, edge buffering and data-quality indicators.
Alert fatigue / false alarms	Configurable thresholds and model tuning with technician feedback.
Cyber-security	Role-based access, encryption, segmentation and audit logging.
Change resistance	Agile pilots, user involvement and clear, simple interfaces.

6.3 Risks and Implementation Challenges

Table 14: Risk register

Risk	Impact	Mitigation
Poor or insufficient historical failure data	Low initial prediction accuracy	Start with rule-based thresholds; learn from data over time; use vendor data.
Integration with legacy machines	Delays and extra cost	Retrofit sensors and gateways; prototype early; use standard protocols.

Risk	Impact	Mitigation
False alarms or missed failures	Loss of user trust	Tune models; review feedback in retrospectives; tiered alert levels.
Cyber-attacks on connected equipment	Data loss or production risk	Network segmentation, encryption, patching, penetration testing.
Staff resistance and skill gaps	Low adoption	Training, pilot success stories, simple interface, management support.
High initial investment	Budget pressure	Phased rollout beginning with critical assets; track ROI.
Network or cloud outage	Interrupted monitoring	Edge buffering, redundancy and failover.
Scope creep	Schedule overrun	Prioritised backlog and product-owner control.

6.4 Limitations

32. Prediction quality depends on the quality and volume of sensor and maintenance data; accuracy will improve gradually and cannot be guaranteed from the start.
33. Not every failure mode can be detected by sensors, so some sudden failures may still occur.
34. Older equipment needs retrofitting, which increases cost and effort.
35. The platform depends on reliable connectivity and skilled staff to configure and maintain policies and models.
36. Benefits will appear over time, not immediately, and require organisational change as well as software.

6.5 Possible Future Enhancements

37. **Digital twins** of critical machines for simulation and what-if analysis.
38. **Advanced AI models**, such as deep learning for vibration and acoustic analysis, and automatic model retraining.
39. **Augmented reality** guidance for technicians during repairs.
40. **Spare-parts optimisation** linked to predicted failures and supplier lead times.
41. **Energy and sustainability analytics** to reduce energy use and waste.
42. **Edge AI** to run models directly on gateways for faster response and offline operation.
43. **Vendor and supplier portals** to share condition data and speed up warranty and service support.

6.6 Conclusion

The proposed Predictive Maintenance Platform replaces reactive and calendar-based maintenance with a data-driven, condition-based approach. Its modular, scalable and secure design addresses the challenges faced by every stakeholder, and the Agile Scrum with DevOps methodology allows it to be delivered in manageable increments and improved continuously. Despite limitations around data quality and integration, the platform offers a practical route to less downtime, lower cost, longer equipment life and more efficient

smart manufacturing.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Understanding (30%)	Clearly explains the problem, objectives, and scope with complete understanding.	Explains the problem with minor omissions.	Basic understanding with limited explanation.	Poor understanding; objectives and scope are unclear or missing.
Stakeholder & Challenge Analysis (25%)	Identifies all stakeholders, their roles, and challenges with clear analysis.	Identifies most stakeholders and challenges with adequate analysis.	Identifies only a few stakeholders or challenges.	Stakeholders and system analysis are incomplete or inaccurate.
Current System Analysis (20%)	Thoroughly analyzes the existing system, strengths, weaknesses, and identifies all gaps.	Good analysis with most strengths and weaknesses identified.	Limited analysis with partial identification of issues.	Proposed solution is unclear or inappropriate; requirements are largely missing.
Proposed Software Solution & Software Engineering Principles (15%)	Proposes an innovative, feasible solution applying appropriate software engineering principles (modularity, scalability, maintainability, security, etc.).	Suitable solution with good application of software engineering principles.	Basic solution with limited application of software engineering principles.	Methodology is inappropriate, poorly justified, or not discussed.
Methodology/ Framework Justification (10%)	Selects the most suitable SDLC/model/framework and provides strong technical justification.	Suitable methodology selected with adequate justification.	Methodology selected but justification is weak.	Outcomes, limitations, or future enhancements are missing; report lacks organization and clarity.

Criteria	**Mark**
Problem Understanding (30%)	/5
Stakeholder & Challenge Analysis (25%)	/5
Current System Analysis (20%)	/5
Proposed Software Solution & Software Engineering Principles (15%)	/5
Methodology/ Framework Justification (10%)	/5
TOTAL	/25